PalletDiT: Code Guide

Manuj Naman, with Claude (Anthropic) as implementation assistant
Anantak Robotics
2026-10-24 (second edition)

How the `pallet/` package works, file by file and tensor by tensor, and how to run, evaluate, extend and reproduce it. Companion to `PALLET_POINTDIT_PAPER.md` (the what and why) and `PALLET_POSE_HANDOVER.md` (the running log). Everything lives on branch `pallet-pose`; the upstream PointDiT files are untouched.

I. Map of the package

file	role
`pallet/targets.py`	the 7 channels, fixed encoding, padding, pixel rays, reconstruction from (range, pose)
`pallet/dataset.py`	`PalletDataset` over the manifest; masks; splits; on-the-fly training priors (`PriorConfig`)
`pallet/compact.py`	full ray tensors -> compact records (cls, range, pose) with verification
`pallet/manifest.py`	manifest (one row per image) from the training store
`pallet/make_split.py`	split.csv for THIS machine's manifest from a pick list (same picks on laptop and ml-02)
`pallet/model.py`	`PalletNet`: PointDiT with 6 in / 6+4 out, flow + class + consistency loss, warm-startable `generate`, checkpoint loading
`pallet/pose.py`	rotation by Kabsch from (theta, phi), camera-centre votes
`pallet/metrics.py`	per-image metrics (3D, lateral/depth/yaw, centre-in-camera, bearing), aggregation by face distance
`pallet/train.py`	training loop (bf16, EMA, cosine, validation, panels, resume, prior-conditioned training)
`pallet/evaluate.py`	evaluate a checkpoint: steps, hybrid, rotation source, a fixed split from another run or machine
`pallet/sequence.py`	sequential estimation: prior rendering from the previous frame, pose propagation/fusion, warm-start evaluation
`pallet/yawfit.py`	per-pick yaw fit experiments (both objectives are blind to a global yaw; kept as the negative result)
`pallet/visualize.py`	inspection panels in the pipeline's colour scheme
`pallet/plot_runs.py`	fit-statistics chart from the logs
`pallet/splits/`	pick lists: `fixed_val_picks_2026-10-19.csv` (201 clean picks), `prior2_picks_2026-10-20.csv`
`pallet/scripts/`	data feeders (ray loop, prefetch, puller, compactor), day/night chains, `md2pdf.js`
`pallet/tests/`	12 tests in 7 files: round trips, geometry identities, pose recovery, compact records, priors, consistency, GPU smoke

Run anything with the repo venv from the repo root: `.venv/bin/python -m pallet.<module>`; tests: `.venv/bin/python -m pytest pallet/tests -q`. Two environment variables point PointDiT at the DINOv3 code and weights: `DINOV3_REPO=$PWD/third_party/dinov3` `DINOV3_WEIGHTS_DIR=$PWD/pretrained/dinov3` (both are symlinks into `~/Anantak/TensorRT-Models`).

II. Frames, conventions and the data flow

Frames. Camera: x right, y down, z forward, pinhole with f ~317–319 px, no distortion, principal point (120, 212) in the upright 240x424 image. Pallet: origin at the centre of the front face on the ground, x lateral (along the face), **y down** (gravity is exactly +y), z into the pallet. `pallet_P_cam` = (R row-major 9, p 3) maps camera to pallet: X_pal = R X_cam + p; p is the camera centre in the pallet frame (`cam_in_pallet`). The pallet origin in the camera frame is $-R^T p$.

Upright image. The raw PNG is 424x240 landscape; `rot_code 2` = `np.rot90(raw, 1)` gives the 424 (H) x 240 (W) upright image the targets are rendered in. Padding adds 4 rows top and bottom: 432x240, the ViT patch is 16 px, cy becomes 216.

```
raw PNG 424x240 (landscape)
  -> rot90 -> upright 424 (H) x 240 (W)
  -> pad 4 rows top + 4 bottom -> 432 x 240

ray record (npz, full or compact)
  -> targets.encode / targets.reconstruct
  -> cont [6,424,240], valid [6,424,240], cls [424,240]
  -> pad -> [6,432,240] / [432,240]

PalletDataset.__getitem__ returns
  image        [3,432,240] f32 in [0,1]
  target       [6,432,240] f32 (0 where invalid)
  valid_mask   [6,432,240] bool
  cls          [432,240]   int64 (255 = padding)
  pixel_mask   [432,240]   bool
  intrinsics   [4] fx, fy, cx, cy of the padded image
  pallet_P_cam [12] R row-major then p
  cam_in_pallet [3];  face_dist_m, key, scan
  (+ prior [6,432,240], prior_mask, has_prior when PriorConfig is set)

PalletNet.forward(batch)   -> loss
PalletNet.generate(image)  -> x_pred [B,6,H,W], logits [B,4,H,W]
targets.decode(x_pred, m)  -> P [3,H,W], range, theta, phi (m, rad)
pose.estimate_pose(...)    -> R, c (camera centre in the pallet frame)
```

Positional embeddings are interpolated by PointDiT itself from the pretraining 16x16 grid to 27x15 patches (`model.py: interpolate_pos_encoding`); nothing else in the backbone depends on the image size.

III. The targets (`targets.py`)

Channel order of the continuous tensor is `CHANNELS = (px, py, pz, range, theta, phi)`; `HIT_CHANNELS = slice(0, 4)` and `RAY_CHANNELS = slice(4, 6)` name the two validity groups. Classes: `CLS_PALLET=0, CLS_LOAD=1, CLS_GROUND=2, CLS_BG=3, CLS_IGNORE=255`.

Encoding (`encode`): stack P, range, theta, phi; `valid = isfinite`; replace NaN by 0; apply the fixed affine `enc = (raw - TARGET_OFFSET) / TARGET_SCALE` with

`TARGET_OFFSET = (0, 0, 0, 0, π/2, π/2)` `TARGET_SCALE = (2, 2, 4, 4, 0.5, 0.5)`

The scales are roughly the per-channel standard deviations over hit pixels within 12 m (px 1.65, py 1.8, pz 4.5, range 4.5 m; angles 0.3 rad around π/2), chosen once on 2026-10-15 so the network sees O(1) values with noise of scale 1. `decode` inverts it and restores NaN where the mask is false. **There is no per-frame centring or scaling anywhere.** The angles are centred on "straight at the face" because that is where every approach ends.

Geometry helpers. `pixel_rays(fx, fy, cx, cy, H, W)` returns the unit ray of every pixel centre ((u+0.5-cx)/fx, (v+0.5-cy)/fy, 1) normalised. `targets_from_pose(rays, range, pallet_P_cam)` applies

`d = R d_cam,  P = p + range * d,  theta = arccos(d_y),  phi = atan2(d_z, d_x)`

and is the single source of truth used by the tests, the pose voting and the compact reconstruction. `reconstruct(cls, range, pose, intrinsics)` builds the full target dict from a compact record; P equals the pipeline's P to < 1 mm on pallet/load/ground pixels (verified in `tests/test_compact.py`); background pixels become on-ray points instead of splatted lidar points.

IV. Masking, in detail (`dataset.py`, `model.py`)

There are four kinds of "invalid" and each is handled explicitly:

1. **No hit / unknown range** (NaN in the pipeline output, e.g. sky through a dock door): `valid_mask[0:4] = False`, target 0. theta/phi stay valid: they are ray geometry, defined for every pixel.
2. **Too far** (`--max_range_m 12`, default): P/range validity is cleared for hit pixels beyond 12 m (13 % of hit pixels, almost all far ground and background). Their class is still supervised.
3. **Padding rows**: `pixel_mask = False`, all `valid_mask = False`, `cls = 255`, image black.
4. **Background P** is kept as a target (it is a real, if slightly off-ray, 3D point) but never used for pose voting.

In the loss (`PalletNet.forward`):

```
e = randn_like(x) * noise_scale
x_in = where(valid, x, e)                          # invalid entries are pure noise at
z = t * x_in + (1 - t) * e
se = (x_pred - x)**2 * channel_weights / (1 - t).clamp_min(t_eps)**2
flow = se[valid].mean()                            # masked per channel, not per pixel
ce = F.cross_entropy(logits, cls, ignore_index=255)
```

Upstream PointDiT uses one `[B,H,W]` mask repeated over 3 channels (`denoiser.py:115-142`); ours is `[B,6,H,W]` so that angles can be supervised where P cannot. In the metrics, pose votes use `cls_pred ∈ {pallet, load}` on real pixels; the rotation uses all real pixels. The `1/(1-t)²` weight is exactly PointDiT's v-space MSE rewritten for x-prediction, so the objective and the pretrained weights agree.

V. The dataset (`dataset.py`)

- `load_manifest(path, picks=None, split=None)` reads `images.parquet`, keeps rows with a tensor, drops `not_a_pick`, and **re-checks that image and tensor exist on disk** (the raw image folders on the laptop are a rolling window).
- `assign_splits(df, val_frac, seed)` splits **by pick**, stratified by `kind_id` (unknown kind is its own group) and topped up at random to `round(val_frac * n_picks)` validation picks. The split is saved as `split.csv` in each run directory. For comparisons across runs and machines use a fixed pick list instead (section IX).
- `PalletDataset(manifest_or_df, picks, split, crop, color_jitter, max_range_m, pad, prior)`: `load_raw(i)` opens the PNG, rotates it upright, loads the npz (full or compact; a compact record has no `P` key and is reconstructed), reads `cameras.json` of the pick for the intrinsics (cached per pick). `__getitem__` encodes, applies `max_range_m`, pads, optionally crops (multiples of 16; cx/cy shifted exactly), converts to tensors. No flip, no normalisation.
- `decode_sample(sample)` un-pads and decodes back to the npz layout, used by the round-trip test and by the panels.
- **Training priors** (`prior=PriorConfig(...)`): for sample i the dataset picks a frame 1..`back` earlier in the same pick, renders its GT pallet/load pixels plus the visible ground plane into the current view (`sequence.render_prior`) after corrupting the pose (rotation N(0, `rot_deg`) about a random axis, translation N(0, `trans_cm`)), adding per-pixel P noise (`p_noise_cm`), a per-frame P bias (`p_bias_cm`) and 16-px block dropout (`dropout`). Returns `prior` [6,H,W] (encoded), `prior_mask` and `has_prior` (False for a pick's first frame). Defaults: back 3, 1 deg, 5 cm, 4 cm, 4 cm, 0.2.

VI. The model (`model.py`)

```
self.net = PointDiT_models["PointDiT-B/16"](input_size=256, in_channels=6, feature_embe
self.net.out_channels = 6 + 4
self.net.final_layer.linear = nn.Linear(hidden, 16*16*10)   # zero-initialised, as upst
```

That is the whole architectural change. `unpatchify` reads `self.net.out_channels`, so the forward returns `[B,10,H,W]`; channels `0:6` are the x-prediction, `6:10` the class logits. The conditioning path (frozen DINOv3-B/16, four intermediate layers concatenated to 3,072 channels, concat with the point tokens, `concat_proj`) is bit-for-bit the pretrained one.

Loading weights. `load_pretrained(path, use_ema=True)` accepts the released checkpoint (`model_ema1`) or a `train.py` checkpoint (`ema`), drops keys whose shapes differ — exactly `net.x_embedder.proj1.weight` (`[128,3,16,16]` vs `[128,6,16,16]`) and `net.final_layer.linear.{weight,bias}` — and asserts nothing else is missing except DINOv3 (`net.y_embedder.*`, loaded by torch.hub from `DINOV3_WEIGHTS_DIR`). `trainable_parameters()` excludes the frozen `net.y_embedder`.

Forward (training). `forward(batch, prior_p=0.0)` samples `t = sigmoid(N(P_mean, P_std))`, sets `t = 0` with probability `zero_t_ratio`, builds `x_in` from the target (noise where invalid), and, if `prior_p > 0` and the batch carries priors, replaces `x_in` by the prior (noise where unknown) for a fraction `prior_p` of the samples that have one. The loss is always against the current target: flow + `cls_weight` x CE + `consist_weight` x consistency.

Consistency loss (`consistency`): mean over supervised hit pixels of |P_pred − (c_gt + range_pred R_gt d)|, in metres, with the pixel rays rotated into the pallet frame by the GT rotation (`pallet_rays`). It ties the P and range channels to the same point on the ray; before it, they disagreed by ~15 cm median, and the vote `c = P − range R d` paid for it. Weight 0.5 in prior2.

Inference. `generate(image, steps=1, x_prior=None, prior_mask=None, t0=0.0, prior_fill="noise", generator=None)`: cache the DINOv3 features once, start at `z = 0, t = 0` (single frame) or at time `t0` from the prior (section VIII b), then Euler steps on the x-prediction `z <- z + (t_next − t)(x_pred − z)/(1 − t)`, the last step returning `x_pred`. With one step and no prior it is a plain regressor: one forward pass. Returns encoded `x_pred` and the logits of the last step.

Image path: dataset gives [0,1]; `forward`/`generate` map to [-1,1] as upstream `engine.py:97`; PointDiT re-maps to ImageNet mean/std before DINOv3 (`model.py:575`). Mixed precision: `torch.autocast(bfloat16)` around forward and generate; losses in float32.

VII. Training (`train.py`)

Flags, grouped.

group	flags (defaults)
data	`--manifest` (store manifest), `--picks`, `--val_frac 0.2`, `--split_seed 0`, `--split_csv` (use a given split instead), `--max_range_m 12`, `--crop`, `--color_jitter`
init	`--pretrained` (B/256 public or a run's checkpoint, EMA weights), `--no_pretrained`, `--resume` (model + EMA + optimizer + step + epoch), `--model PointDiT-B/16`, `--feature_embedding_type dinov3_vitb16`
objective	`--zero_t_ratio 0.1`, `--noise_scale 1.0`, `--cls_weight 1.0`, `--consist_weight 0.0`, `--channel_weights` (6 values)
priors	`--prior_p 0.0` (fraction of samples trained from a prior), `--prior_back 3`, `--prior_rot_deg 1`, `--prior_trans_cm 5`, `--prior_noise_cm 4`, `--prior_bias_cm 4`, `--prior_dropout 0.2`
optimisation	`--batch_size 8`, `--epochs 60`, `--lr 1e-4` (absolute), `--min_lr 1e-6`, `--warmup_steps 300`, `--weight_decay 0`, `--grad_clip 1.0`, `--ema_decay 0.999`
validation	`--eval_every 5` (epochs), `--eval_batch_size 8`, `--val_max_images 600` (even subsample), `--val_raw`, `--n_panels 6`, `--steps 1`
misc	`--num_workers 6`, `--log_every 20`, `--seed 0`, `--max_steps` (smoke tests), `--out`

Loop. AdamW with betas (0.9, 0.95) as upstream, per-iteration linear warmup then cosine, gradient clipping, non-finite loss skips the step. EMA: a deep copy updated every step with decay `min(ema_decay, (1+step)/(10+step))`; validation and the saved `ema` weights use it. Validation runs `generate` on the held-out subsample, `metrics.image_metrics` per image, `aggregate` medians per face-distance bin, prints the product table and the full table, saves `val_<epoch>_<ema|raw>.json`, writes scalars to tensorboard and 6 prediction-vs-GT panels. `checkpoint-last.pth` = {model, ema, optimizer, step, epoch, args}.

Throughput (RTX 3080 Ti laptop, 16 GB): 18–20 img/s at batch 16 (8.6 GB), ~15 img/s at batch 8; with on-the-fly priors ~17 img/s at batch 16 (the dataloader renders priors on the CPU, 8 workers). ml-02 (RTX 5060, 8 GB): batch 8 only, ~10 img/s with priors, ~20 img/s without.

Recipe of every run (all PointDiT-B/16 at 432x240, frozen DINOv3-B, 1-step validation; "clean" = overall median 3D origin error on the 201-pick fixed split, which no run trained on):

run	init	train picks	epochs x batch, lr	extras	clean (cm)
b256_baseline	public B/256	30	60 x 8, 1e-4		—
b256_run2	run 1	318	12 x 16, 1e-4		—
b256_run3	run 2	655	12 x 16, 7e-5		12.5
b256_run4	run 3	816	10 x 16, 5e-5		12.0
b256_run5	run 4	1,453	8 x 16, 4e-5		11.1
b256_run6	run 5	2,030	8 x 16, 3e-5		10.5
b256_prior1	run 6	2,030	3 x 16, 3e-5	prior_p 0.5 (1 deg / 5 cm)	9.9
b256_prior2	run 6	2,030	3 x 16, 3e-5 (epoch 3 at 8, 1.5e-5 on ml-02)	prior_p 0.5 (2 deg / 8 cm), consist 0.5	**8.0**
b256_indep	public B/256	all but fixed_val	2 x 8, 5e-5	lineage test only	16.0

Typical launch (always detached, so a dropped terminal does not kill it):

```
export DINOV3_REPO=$PWD/third_party/dinov3 DINOV3_WEIGHTS_DIR=$PWD/pretrained/dinov3
setsid nohup .venv/bin/python -m pallet.train --out runs/<name> --pretrained runs/<prev
    --batch_size 16 --lr 3e-5 --warmup_steps 300 --epochs 3 --eval_every 1 --val_raw \
    --prior_p 0.5 --prior_rot_deg 2 --prior_trans_cm 8 --consist_weight 0.5 > runs/<nam
```

To resume after a stop: add `--resume runs/<name>/checkpoint-last.pth` and repeat the other flags (the schedule is per step and continues; `--split_csv runs/<name>/split.csv` keeps the same picks).

VIII. Inference, pose and metrics (`model.generate`, `pose.py`, `metrics.py`)

The vote, worked through. For a pixel with camera ray d (unit), predicted pallet-frame point P, predicted range r and the rotation R: the camera centre must be c = P − r (R d). Every pallet/load pixel gives one such estimate; the median over 10^3–10^4 of them is robust to a wrong class here and there. The rotation comes first, from the angles: each real pixel's predicted (theta, phi) is a direction in the pallet frame, d_pal = (sin θ cos φ, cos θ, sin θ sin φ), and its camera ray d is known, so R is the Kabsch solution of min $\Sigma|d_pal - R\,d|^2$ over ~10^5 pixels (SVD of the 3x3 cross-covariance, determinant-corrected). On ground-truth targets this recovers c to < 1 mm and R to < 0.001 deg (`tests/test_pose.py`). `estimate_pose(P, rng, theta, phi, rays_cam, vote_mask, ray_mask, R=None)` returns R, c (median), c_mean, spread (median of |vote − c|) and n_votes.

Per-image metrics (`image_metrics(sample, x_pred, logits, rot_from="all")`), medians aggregated per bin by `aggregate` (bins 0–2, 2–3, 3–4.5, 4.5–6, 6–8, 8–12 m and "all"):

metric	meaning
`cls_acc`, `iou_<class>`	segmentation on real pixels
`P_err_cm`, `range_err_cm`	median over GT pallet/load pixels
`theta_err_deg`, `phi_err_deg`	median over real pixels
`rot_err_deg`, `rot_err_vec_deg`	geodesic angle (Frobenius form, well conditioned) and its signed axis-angle vector in the pallet frame
`origin_err_cm`, `origin_err_xyz_cm`	

metric	meaning
`lateral_err_cm`, `depth_err_cm`, `yaw_err_deg`	the product metrics: abs x and abs z of the origin error, abs rotation about pallet y
`centre_cam_lateral_err_cm`, `centre_cam_depth_err_cm`, `bearing_err_deg`	the pallet origin seen from the camera, $-R^T c$: yaw-independent steering quantities
`origin_err_gtmask_cm`	the vote with the GT class mask and GT rotation: an upper bound on what voting could do with perfect segmentation
`vote_spread_cm`, `n_votes`	dispersion and count of the votes
`fused_*`	the same pose metrics after re-voting with the fused rotation (sequence mode only)

`format_table(agg)` prints the product table first (lateral, depth, yaw, origin, centre-in-camera lateral, bearing) and then the full table. **Known relations**: lateral error ≈ distance x yaw error (r 0.98); the yaw error is a per-pick label bias of ~1.3 deg; the centre-in-camera lateral error is 1.5–3.3 cm at every distance.

`evaluate.py` evaluates a run's EMA (or raw, `--raw`) checkpoint on its validation rows: `--steps 1 3 5`, `--hybrid k` (angles and class from 1 step, P/range from k steps), `--rot-from obj` (rotation from pallet/load pixels only), `--val_max_images`, and `--split-from <dir>` to use another run's or a fixed `split.csv`. Output: `eval_<config>_<tag>.json` in the run directory and the tables on stdout. Findings so far: 3 steps sharpen P (−1.5 cm) but worsen the angles; rotation from object pixels is worse.

VIII b. Sequential estimation (`sequence.py`)

Why it is possible. The targets live in the static pallet frame, so the previous frame's P is a prediction for the next frame (only the pixel correspondence changes), the angles follow exactly from the propagated pose, and the flow objective trained the network at every t, so it can start denoising from a prior at time t0 instead of from zeros at t = 0.

Pose propagation. With the inter-frame motion dT ($X_{k+1} = A X_k + b$; from the recorded poses here, from odometry on the unit): $R' = R A^T$, $c' = c - R' b$ (`relative_motion`, `propagate_pose`).

Prior rendering (`render_prior(P, rng, cls, valid, R, c, intrinsics, H, W, splat=3, ground=True, angles_only=False, ground_side_m=0.9)`): the previous frame's selected pixels are transformed to the new camera ($X_{cam} = R^T(P - c)$), projected, z-buffered with a 3-px splat (P unchanged, range = $|X_{cam}|$); theta/phi come from the new rays and R; the ground plane y = 0 is drawn analytically where it is visible (in front of the face, $z < -0.05$, or beside the pallet, $|x| > 0.9$ m) and nothing nearer was rendered. Everything else is unknown (mask false). The output is an encoded `cont [6,H,W]` and `mask [6,H,W]`.

Warm start. `generate(..., x_prior, prior_mask, t0, prior_fill)`: `noise` (default) builds `z0 = t0 (prior where known, e elsewhere) + (1 − t0) e`, exactly the training distribution; `mixed` keeps known entries at the zero-noise mean `t0 prior`; `zeros` is wrong (an encoded 0 is "a point at the pallet origin") and kept only for reference.

`run_pick` walks a pick in time order and, per frame, builds the prior from the chosen source, runs the warm start, votes, updates the fused pose and records metrics:

option	meaning
`--sources`	`pred` own maps + own fused pose; `pred_gtpose` own maps + GT pose; `oracle` previous GT maps + GT pose; `oracle_predpose`; `gtcur` current GT at t0 (ceiling); `pose`, `pose_gtpose` angles only
`--t0`	start time(s); `--steps` Euler steps from t0 (1 is best)
`--t0_far D_NEAR D_FAR`	scale t0 from full at ≤ D_NEAR m to 0 at ≥ D_FAR m, using the previous frame's estimated distance
`--pose_alpha`	weight of the newest vote in the fused pose (EMA on R via SVD re-orthonormalisation, linear on c), propagated with dT each frame; 1 = raw vote
`--consistency_cm`	keep only own-map pixels whose P and range agree along the ray (0 = off; 5 cm removed every pallet pixel)
`--prior_channels maps`	prior for P/range only, angles left unknown (worse: the net needs the angle prior)
`--prior_fill`, `--splat`, `--prior_ground`	as above
`--picks N`, `--split`, `--run`, `--out`	N evenly chosen validation picks of the split; the checkpoint; output dir with one JSON per configuration and `summary.csv`

Each per-frame row also carries `fused_*` metrics (the frame's own votes re-projected with the fused rotation) and `smoothed_origin_err_cm`; the summary adds `jitter_cm` (median over picks of the mean frame-to-frame jump of the origin error).

Recommended configuration (2026-10-20): `--sources pred --t0 0.8 --t0_far 3 6 --pose_alpha 0.2 --steps 1 --consistency_cm 0 --splat 3 --prior_fill noise`, with the prior-trained checkpoint `runs/b256_prior2`. On the 201 clean picks it is better than or equal to single-frame everywhere (lateral 3.1 / 4.0 / 6.0 / 9.4 cm below 6 m vs 3.3 / 4.2 / 6.7 / 9.8; yaw 0.84 vs 0.90 deg). With a correct pose the same machinery reaches 4.4 cm overall and 5–8 cm beyond 6 m; the difference is the heading.

Training with priors is section V (dataset) + section VI (forward); `tests/test_prior_training.py` checks that an uncorrupted prior matches the next frame's GT and a corrupted one stays in range.

IX. Splits, two machines and reproducibility

- **Never compare models on per-run splits**: each run re-splits a growing store, so later runs trained on earlier validation picks (run 6 on run 5's split looked 1 cm better than it was). The clean set is the 201 picks in `pallet/splits/fixed_val_picks_2026-10-19.csv`, which no run has trained on; keep them out of training.
- `python -m pallet.make_split --manifest <images.parquet> --picks <pick csv> --out <dir>/split.csv [--default train|drop]` turns a pick list into a split file for the local manifest (a pick csv without a `split` column marks its picks `val`). `evaluate.py --split-from <dir>` and `sequence.py --split <dir>` read `<dir>/split.csv`; `train.py --split_csv` trains on it.
- **ml-02** (`ubuntu@192.168.5.193`, RTX 5060 8 GB): the repo lives at `~/Libraries/GoogleResearch/pointdit` with its own venv, the public B/256 weights, run 6 / prior1 / prior2 / indep checkpoints, and `runs/fixed_val/split.csv` built for its manifest (`~/Anantak/Data/PalletModels/manifest/images.parquet`, SensorUnit's `pallet_manifest.py`, 3,341 picks; full ray tensors). Sync code with `rsync -a --exclude .venv --exclude runs --exclude pretrained --exclude third_party/dinov3 --exclude __pycache__ pointdit/`

ml-02:~/Libraries/GoogleResearch/pointdit/; copy checkpoints explicitly. Batch 8 fits; batch 16 does not.

- **Reproduce the headline numbers** (paper section VIII), on either machine:

```
.venv/bin/python -m pallet.evaluate --run runs/b256_prior2 --steps 1 --split-from runs/
.venv/bin/python -m pallet.sequence --run runs/b256_prior2 --split runs/fixed_val --pic
    --t0 0.8 --t0_far 3 6 --pose_alpha 0.2 --steps 1 --consistency_cm 0 --splat 3 --pri
```

X. Data tooling (`compact.py`, `manifest.py`, `scripts/`)

- **Training store** `~/Anantak/Data/PalletTrainImages/<robot>/<pick>/{02/color_NNNN.png, rays/cameras.json, rays/cam2_NNNN.npz}`: images plus **compact records** (`cls, range, pallet_P_cam, cam_in_pallet, scan, ts_img_us`, ~250 KB per image instead of 1.4 MB). `compact_pick(src_rays, dst_rays)` verifies the reconstruction against the source before writing and never deletes unless asked (`--delete-src`). The laptop store held 2,620 picks (~77k images) on 2026-10-20.
- **Manifest.** `python -m pallet.manifest` walks the store's `cameras.json` files and writes `manifest/images.parquet` with the columns the dataset uses; kinds come from SensorUnit's `picks.parquet`. On ml-02 the SensorUnit script is used instead (full tensors under `PalletModels`, images under `PicksCopies`).
- **Feeders** (all `setsid nohup`, logs in `~/Anantak/Data/PicksCopies/_logs/`): `rays_from_ml02.sh` builds ray tensors on the laptop GPU from raw episodes on ml-02 (waits while the GPU holds > 2 GB), `prefetch_from_ml02.sh` stages raw episodes, `pull_ml02_rays.sh` / `pull_loop.sh` copy picks already built on ml-02 (no GPU), `compact_loop.sh` folds new tensors into the store every 10 minutes. The `day*`, `night*` and `ml02_chain_*` scripts are the exact job chains of 2026-10-15..20.
- **Documents.** `node pallet/scripts/md2pdf.js docs/X.md docs/X.pdf` reproduces the Markdown PDF plugin's output (fenced divs, Letter, the user's margins and page numbers) with the plugin's own renderer and Chrome.

XI. Reading the outputs

- `runs/<name>.log`: one line per 20 steps — loss, flow, class CE, consistency (`cons`, metres), per-channel RMSE in raw units, lr, grad norm, img/s, memory; validation tables every `eval_every` epochs (product table first).
- `runs/<name>/`: `val_*.json` (per-image rows + aggregate), `eval_*.json` (from `evaluate.py`), `panels/` (GT panel over prediction panel, caption with the origin error), `tb/` (tensorboard), `split.csv`, `args.json`, `checkpoint-last.pth`.
- `runs/seq_*/`: one JSON per configuration with per-frame rows, `summary.csv` with jitter, per-bin origin, lateral and fused lateral.
- `runs/fit_stats.png` (`plot_runs --loop 300`) and tensorboard on port 6006 (`tensorboard --logdir runs --port 6006 --bind_all`).
- Panels colour code: pallet/load pixels by pallet-frame position on a fixed metre scale (R = x −0.75..0.75, G = height 0..2, B = depth 0..1.3), range 0–8 m, theta, phi, class masks (pallet orange, load blue, ground green, background-with-range grey).

XII. Pitfalls met so far

- **Rolling raw window.** `~/Anantak/Data/PicksCopies` keeps only the 60 newest raw pick folders per robot; images vanish. The store is the stable copy; the dataset checks file existence.

- **GPU sharing.** The SensorUnit pipeline app is CUDA and runs out of memory next to a training run; building and training alternate.
- **Waiting on process names.** Two multi-hour stalls came from `pgrep -f <pattern>` matching a shell whose command line merely contained the pattern (a wrapper shell, a heredoc). Wait on files, PIDs, `pgrep -x`, or GPU memory instead.
- **Validation set drift and leakage.** See section IX; use the fixed split.
- **float32 rotations.** The stored `pallet_P_cam` is float32; use the Frobenius-based angle, not arccos of the trace.
- **Priors and zeros.** At t > 0 an encoded 0 means "a point at the pallet origin"; never feed zeros where a prior is unknown.
- **Splatting predictions.** Only splat predicted pallet/load pixels into a prior; predicted background/ground points land in front of the pallet. Draw the ground plane analytically and only where it is visible.
- **The net ignores noise only where it was never supervised.** In training, noise in the input at intermediate t marked unsupervised pixels, so an untrained-for partial prior wrecks the uncovered pixels; prior-conditioned training fixes it.
- **Multi-step from a prior diverges far away.** One Euler step from the prior; fade the prior with distance.
- **Yaw is not observable from the sequence geometry.** Trajectory consistency and vote dispersion are both invariant to a global yaw; the heading must come from the image (or from corrected labels).
- **Metric ordering.** `image_metrics` derives lateral/depth/yaw from `origin_err_xyz_cm` and `rot_err_vec_deg`; keep those computed first (a reorder once crashed every ml-02 run).